

One Cluster, Every Vendor: Cross-Vendor Tensor-Parallel LLM Inference over Commodity Ethernet

Hanzo AI — ML Systems

Distributed companion to the *One Source, Every Backend* series — extends Paper 05
across the wire

Abstract

Real accelerator fleets are vendor-mixed: an AMD APU, an NVIDIA dev kit, and a Mac sit on the same desk, and no collective-communication library spans them — NCCL is NVIDIA-only, RCCL is AMD-only, and neither will open a channel to the other. This paper reports a working result, with its limits stated plainly. One inference engine (**hanzo-engine**, Rust, one codebase, per-backend kernels) runs on three vendor-diverse accelerators — AMD ROCm gfx1151 (a “Strix Halo” APU, node *evo*), NVIDIA CUDA sm_121 (a GB10 Blackwell, node *spark*), and Apple Metal M4 Max (node *dbc*). We make its Megatron-style tensor-parallel (TP) collectives *vendor-agnostic* by host-staging every exchange — `device`→`CPU`→`TCP`→`CPU`→`device` — so any device that can copy to host storage can join the ring, which is exactly what NCCL/RCCL cannot do across a vendor boundary. On a two-node ring (*evo* ROCm + *spark* CUDA) over a 2.5GbE direct link, the sharded engine **builds, connects, shards a world_size=2 tensor-parallel model, and generates fully coherent multi-token output — chain-of-thought and a correct answer — across the AMD/NVIDIA boundary at 50.4 tok/s decode** (TTFT 0.09s, 213 tok/s prefill). To our knowledge this is the first reported cross-vendor (AMD+NVIDIA) tensor-parallel LLM inference over commodity ethernet in a single engine.

We are equally explicit about the engineering, because the honest failures are the paper’s most useful content. The socket-ring path had only ever been *compiled*, never *run* (only the NCCL path was exercised), and bringing it up cross-vendor surfaced four heterogeneity bugs — a `1en2` byte-count overflow, a missing ROCm host-staging arm that meant the ring never even compiled for AMD, a 10s handshake timeout that raced heterogeneous startup, and — the subtle one that blocked coherent decode — a *heterogeneous-dtype-on-the-wire* hazard, where the ROCm rank auto-selects `f16` and the CUDA rank auto-selects `bf16` and each reinterpreted its neighbour’s 16-bit words as the wrong floating-point layout, corrupting the reduction from the first layer. The cure is a canonical `f32` ring wire (a lossless superset of both) plus a `world_size-1` accumulating all-reduce correct for any $N \geq 2$ and the removal of a spurious power-of-two world-size restriction; all four fixes are merged on `main`, and a same-vendor control (both ranks `f16`) matches the cross-vendor run token-for-token, isolating the wire dtype as the sole cross-vendor variable. The measured cost of the primitive is itself a result: 50.4 tok/s is $\approx 3.3\times$ slower than the same model single-node (168 tok/s), and the gap is *all* synchronization — an all-reduce every layer, over the wire. That measured tax is the empirical case for **pipeline parallelism** — contiguous layer ranges, activations-only on the wire (`0(nodes)` hops, not `0(layers)` all-reduces), format-agnostic, one generation loop driving stateless workers — as the structurally better primitive for heterogeneous clusters: the numeric fixes made TP *correct*, not well-matched. Three residuals stay honestly open, and we develop them, with five further problems, as a research agenda in the companion [7]: the two ranks still sample independently from their own-dtype logits (a near-tie argmax can diverge and desync the KV caches), the CUDA rank must

run graphless to stay lockstep with the graphless ROCm rank, and TP is still coupled to the dequantized (safetensors) weight path.

1 The problem: fleets are vendor-mixed, and the collective libraries are not

The single-node premise of this series is that the interesting inference target is a commodity box a customer already owns [1]. The multi-node reality is that they own *several*, and not from one vendor. A team that bought an AMD “Strix Halo” mini-PC last quarter, a Grace-Blackwell dev kit this quarter, and codes on a MacBook has three capable accelerators totalling well over a hundred gigabytes of unified memory — enough to hold a model no single one of them can — sitting idle on a LAN. The obstacle is not the models and not the math. It is that *the collective-communication layer is single-vendor by construction*.

Tensor parallelism [8] shards each weight matrix across ranks and, twice per transformer block (after attention, after the MLP), performs an `all_reduce` to sum the partial results back into a full activation. That `all_reduce` is the whole game, and in practice it is delegated to a vendor library: **NCCL** [11] on NVIDIA, **RCCL** [12] on AMD. Both are excellent and both are closed at the vendor boundary: NCCL speaks to NVIDIA GPUs over NVLink/PCIe/GPUDirect and has no concept of an AMD peer; RCCL is the symmetric mirror. There is no NCCL↔RCCL channel, no cross-vendor GPUDirect, and no shared handle either side will accept. So the textbook path to running one model across *evo* and *spark* together does not merely underperform — it does not exist.

The engine already dodges the *first* half of this problem. **hanzo-engine** is one Rust codebase that lowers to CUDA, ROCm, Metal, Vulkan, and CPU behind a single model loader, reaching `llama.cpp` decode parity on all three vendors from that one build [1]; the kernel layer that makes “one source, every backend” hold is the subject of the companion DSL papers [2, 3, 4]. What was missing is the *second* half: a way for those per-vendor engines to talk to *each other* inside one cluster. This paper builds that channel, runs it across a vendor boundary for the first time, and reports — without varnish — exactly how far it gets and what it cost to get there.

2 The design: a host-staged socket ring makes the collective vendor-agnostic

The engine carries a unified `Comm` abstraction with three backends behind one enum: `Nccl` (the fast same-vendor NVIDIA path), `Ring` (a socket-based collective), and `Dummy` (`world_size=1`). The idea that makes cross-vendor work possible is in the `Ring` backend, and it is deliberately humble: **stage every tensor through host memory and move the bytes over TCP**.

Concretely, a ring `sum_all_reduce` does not touch a vendor collective at all. It pulls the tensor’s storage to the CPU, converts it to a canonical `f32` payload, exchanges the bytes with its ring neighbours over ordinary TCP sockets, sums in `f32`, and copies the reduced result back to the device in its native dtype (Listing 1). The single load-bearing property is that the exchange happens in `CpuStorage`, which *every* backend can produce — CUDA, Metal, and ROCm each implement `to_cpu_storage()`. The device that produced the tensor is irrelevant by the time the bytes reach the socket. *Why* the wire dtype is a canonical `f32` rather than the raw device bytes — the difference between coherent and garbage output across vendors — is the subject of §5; here it is simply the shipped design.

Listing 1: The vendor-agnostic collective, condensed from the shipping code. The reduce goes device→host→canonical f32→TCP→f32 sum→device; the `match` over `Storage` is the only place the backend is named, and every arm lands in the same `CpuStorage`. The `cpu_to_f32` stage is what makes an f16 rank and a bf16 rank agree (§5). NCCL, by contrast, would keep the whole exchange on-device and refuse a foreign-vendor peer.

```
fn sum_all_reduce(&self, xs: &Tensor) -> Result<Tensor> {
    let storage = xs.storage_and_layout().0;
    let host = match &*storage { // device -> host (any backend)
        Storage::Cpu(s) => s.clone(),
        Storage::Cuda(s) => s.to_cpu_storage()?,
        Storage::Metal(s) => s.to_cpu_storage()?,
        #[cfg(feature = "rocm")]
        Storage::Rocm(s) => s.to_cpu_storage()?, // was missing: ring never built for AMD (Bug 2)
    };
    let wire = cpu_to_f32(&host)?; // CANONICAL wire dtype: f16/bf16/f32 -> f32
    let sum = self.ring_exchange(&wire, xs.dims())?; // world_size-1 accumulating steps, sum in f32
    Tensor::from_slice(&sum, xs.dims(), xs.device())? // host -> device
        .to_dtype(xs.dtype()) // back to this rank's native compute dtype
}
```

This is not free. NCCL keeps the reduction on-device and rides NVLink; the host-staged ring pays two extra device↔host copies per collective, a widening cast to f32, and then rides a 2.5GbE link whose measured goodput is ≈ 280 MB/s (§7). That is the correct trade for the problem: we spend bandwidth to buy *vendor-agnosticism*, because the alternative on a mixed fleet is not a faster collective, it is *no* collective. The ring is selected purely by configuration — `use_ring()` is true when the `ring` feature is built and a `RING.CONFIG` path is present in the environment (§7) — so the same engine binary is a single-node server, an NCCL rank, or a socket-ring rank depending only on how it is launched.

3 Compiles $\neq$ runs: an unexercised path was broken three ways

Here is the lesson that generalizes past this codebase. *The ring path had been written, had type-checked, had shipped in every build for months — and had never once been run.* All multi-GPU testing had gone through NCCL (the same-vendor path), so the socket ring was dead code that happened to compile. The first time it was asked to carry a real tensor across a real vendor boundary, it failed three separate ways, each a different species of the gap between “the compiler accepted it” and “it does the thing.” All three are fixed on `main` (commit 63cc4c7ac); the fourth, subtler failure — visible only once the generated tokens came out wrong — is §5.

Bug 1 — a `len2` byte count (a logic error the types could not catch). The wire code computed its transfer size as

```
let nbytes = x.len() * std::mem::size_of_val(x);
```

This looks reasonable and is badly wrong. For a slice `x: &[T]`, `size_of_val(x)` is *already* the size of the whole referent — `x.len() × size_of::()` — so multiplying by `x.len()` again yields `len2 × elem.size`, overshooting the true byte count by a factor of `len`. The subsequent `from_raw_parts(ptr, nbytes)` then fabricates a byte slice spanning far past the end of the allocation, and `write_all` walks off the buffer and hits `EFAULT`. It is invisible for a one-element tensor (`len2 = len`) and fatal for every real activation. The fix is one token — delete the `x.len() *` —

but the point is that the type system was content the entire time: both operands are `usize`, the product is a `usize`, and the meaning is nonsense. Only *running* it with `len>1` reveals it.

Bug 2 — the ring never compiled for AMD (a hole hidden by a feature flag). The `device→host match` in Listing 1 had arms for `Cpu`, `Cuda`, and `Metal` but *not* `Rocm`. Because `Storage::Rocm` only exists when the `rocm` feature is enabled, and no one had ever built `ring + rocm` together, the non-exhaustive `match` was never forced into existence. The moment you try to build the one configuration a heterogeneous ROCm+CUDA cluster needs, it fails to compile. The fix adds the two `#[cfg(feature = "rocm")]` arms (one in `sum_all_reduce`, one in `all_gather`) that stage a ROCm tensor to host exactly like the others. This is the subtler lesson: a feature-gated code path is not merely untested, it may be *unbuilt*, and a green CI that never enables that flag combination proves nothing about it.

Bug 3 — a handshake that assumed both nodes were the same speed. The ring’s neighbour-connect loop gave up after 10 seconds. On a homogeneous cluster every rank boots in lockstep and 10s is generous. On a *heterogeneous* cluster the nodes do not load at the same rate — a Blackwell dev kit and an APU parse weights, warm kernels, and reach the barrier at visibly different times — so the faster node would time out waiting for the slower one and abort the whole ring. Raising the timeout to 120s (and matching the panic message) lets asynchronous, uneven startup settle. The assumption “all ranks are interchangeable, so they arrive together” is exactly the kind of invariant that is silently true on one vendor and silently false across three.

None of these three is deep. That is the message. A distributed path can be well-typed, code-reviewed, and shipped, and still be dead on arrival in three independent ways, because “it compiles” certifies none of the things that actually matter at run time — the arithmetic of a byte count, the existence of a match arm under a flag nobody set, or a timing assumption baked into a constant. And a fourth bug, subtler than all three and invisible until the generated tokens themselves came out wrong, waited past these; it is the root cause of cross-vendor incoherence, and its fix is what turns prefill into coherent decode (§5).

4 Result: coherent decode across the AMD/NVIDIA boundary

With the four heterogeneity bugs fixed — three mechanical (§3), one numeric (§5) — the cross-vendor ring generates coherent output. On a two-node ring — *evo* (AMD ROCm gfx1151) as rank 0, *spark* (NVIDIA CUDA sm.121) as rank 1 — connected by a 2.5GbE direct cable, the sharded engine:

1. **builds** with `ring + rocm` on the AMD node and `ring + cuda` on the NVIDIA node (Bug 2 was the blocker);
2. **connects** the ring — each rank binds its listen port, dials its right neighbour, and accepts its left, the handshake surviving the uneven boot (Bug 3);
3. **shards** a Qwen3-0.6B at `world.size=2`, each node holding half of every parallelized weight matrix via the `ColumnParallelLayer/RowParallelLayer` split; and
4. **decodes coherently** — running the host-staged `f32 all_reduce` across the vendor boundary at every layer — emitting a chain-of-thought and a correct final answer in lockstep, at **50.4 tok/s decode** (TTFT 0.09s, 213 tok/s prefill).

The two ranks run *different* 16-bit compute dtypes — *evo* in **f16**, *spark* in **bf16** (§5) — and still agree, because the reduction crosses the wire in canonical **f32**. To our knowledge this is the first reported cross-vendor (AMD+NVIDIA) tensor-parallel LLM inference over commodity ethernet in a single engine: two different-vendor accelerators jointly executing one tensor-parallel forward *and* autoregressive decode, over a commodity cable, with no shared-vendor collective (NCCL/RCCL) between them — the `all_reduce` that both libraries refuse across the boundary runs to completion on plain TCP.

Controls isolate the wire, not the model. A *same*-vendor control — both ranks forced to **f16**, so the raw-bytes shortcut and the canonical-**f32** wire coincide — produces the same tokens as the cross-vendor run, which pins the dtype wire (§5), not the ring mechanism, as the only thing heterogeneity changed. A *single*-node control — the same Qwen3-0.6B on *spark* alone — generates the same answer at 168 tok/s decode and 464 tok/s prefill. So the cross-vendor 50.4 tok/s is $\approx 3.3\times$ slower than single-node decode, and the entire gap is synchronization: a host-staged `all_reduce` every layer, over the wire, is the only added work. The model is coherent and the kernels are at parity; what the ring costs is the wire.

5 Root cause and fix: the heterogeneous-dtype-on-the-wire hazard

The fourth bug was invisible until the tokens themselves came out wrong, and it is the one worth dwelling on, because it is the trap waiting in every host-staged cross-vendor collective. We state the cause, the fix, and the residuals the fix leaves.

5.1 The cause: two ranks legitimately pick different 16-bit formats

The engine picks its compute dtype *per node* from the *local* device. The selector (`determine_auto_dtype_all`) returns **f16** when it sees a ROCm gfx1151 device — RDNA3.5 has fast **f16** matrix cores (WMMA) but no fast **bf16** matmul — and **bf16** for the CUDA Blackwell path. So on a mixed ring the two ranks legitimately and independently choose *different* 16-bit formats: *evo* runs the model in **f16**, *spark* in **bf16**. The original host-staged ring shipped *raw device-dtype bytes* and the receiver reinterpreted them as *its own* local dtype. Since **f16** and **bf16** share a 16-bit container but split it differently — 5-bit exponent / 10-bit mantissa versus 8-bit exponent / 7-bit mantissa — every value crossing the boundary was silently re-read with the wrong exponent/mantissa layout. The bit pattern was intact; its *interpretation* was wrong, which is worse, because nothing errored: the `all_reduce` summed correctly-typed nonsense into correctly-typed nonsense, corrupting activations from the first layer. Notably, a *same*-vendor ring never exposed this — both ranks pick the same dtype and the raw-bytes shortcut accidentally works — which is precisely why the hazard survived months of same-vendor NCCL and ring testing until the first cross-vendor run, and why the same-vendor control (§4) is the clean isolator.

5.2 The fix: a canonical f32 wire and an N-node accumulating reduce

The remedy is structural, not a patch: the wire needs a *canonical* dtype both ranks agree on independent of their local compute dtype. **f32** is the natural choice — a lossless superset of both **f16** and **bf16**, and higher-precision accumulation for the reduction besides. The shipped reduce (Listing 1) stages each device tensor to host, converts to **f32** (`cpu_to_f32`), exchanges and sums

in `f32`, and converts the result back to the receiver’s native device dtype. The 16-bit formats now meet only *inside* each node; the wire is always `f32`, so both ranks reduce the same values bit-for-bit regardless of vendor, and the reduction is carried out in the CPU-`f32` path with a canonical tie-break so a ROCm box and a CUDA box do not diverge on rounding order.

The same pass closed a latent correctness bound. The original `sum_all_reduce` performed a *single* write-right / read-left exchange and summed the neighbour’s shard — a complete all-reduce for two ranks, but for $N > 2$ rank i only ever incorporates rank $i-1$, so ranks $i-2, i-3, \dots$ never reach it. The fix replaces the single exchange with the textbook `world_size-1` accumulating ring (each rank forwards and sums its running shard for $N-1$ steps), correct for any $N \geq 2$, and removes a spurious power-of-two world-size restriction inherited from the NCCL path (`is_power_of_two`) that had made four the smallest cluster past two nodes the ring would even admit. The all-reduce is now algorithmically correct for any $N \geq 2$; the *measured* cross-vendor demonstration in this paper is the two-node *evo+spark* ring, and multi-node ($N > 2$, e.g. adding *dbc/Metal*) cross-vendor runs are the natural next validation, not a claimed measurement here.

5.3 Residuals: three problems the fix leaves open

Coherent cross-vendor decode holds, but three residuals bound the result. We state each crisply and develop all three, alongside five further problems, as a research agenda in the companion [7].

Sampling divergence (a determinism boundary). The *reduced activations* now agree bit-for-bit, but each rank still computes its final logits on its own device in its own 16-bit dtype and samples *locally*. In the demonstrated runs the argmax stays in lockstep; a near-argmax-tie could in principle resolve differently under `f16` versus `bf16` rounding and desync the per-rank KV caches. The robust cure — the head rank samples and broadcasts the token id, so every rank advances on the same token — removes the divergence by construction; it is a determinism-boundary problem in replicated-state distributed inference, not a numeric one.

Graphless lockstep (an execution-symmetry assumption). The two ranks run *two* generation loops kept in phase by the per-token `all_reduce` barrier. To stay in phase the CUDA rank must run *graphless*, matching the graphless ROCm rank and forgoing the $\approx 1.5\times$ decode speedup CUDA-graph capture would give [1]. Decode is coherent this way, but per-token cross-node lockstep between nodes that legitimately want different execution strategies is fragile — an argument for pipeline parallelism (§6), where each node’s execution is a purely local choice, rather than a bug to patch.

Format coupling (a sharding limitation). The TP collectives are wired through the non-quantized (safetensors) model path — `ColumnParallelLayer` and `RowParallelLayer` live in the normal model definitions — and *not* through the GGUF quantized path. So even with the numerics correct, TP cannot yet shard the very block-quantized formats that make edge inference fit in commodity memory. Quant-aware sharding (splitting on superblock boundaries without a dequant-requant loss) is an open problem [7]; pipeline parallelism sidesteps it (§6) because activations, not weights, cross the seam.

6 The case for pipeline parallelism

The dtype hazard and the N-node reduce are fixed, so the tensor-parallel ring on `main` is now *correct*. But correct is not the same as well-matched, and the residuals of §5.3 together with the measured synchronization tax point the same way. Tensor parallelism was built for a homogeneous, tightly-coupled, high-bandwidth, single-vendor machine, and every place it strains on our cluster is a place it assumes that machine: a per-layer all-reduce assumes cheap bandwidth (§2) — the measured $\approx 3.3\times$ decode tax is exactly this bill; per-token lockstep assumes symmetric execution (§5.3); the parallel layers assume dequantized weights (§5.3). Fixing the numerics made the primitive correct; it did not make it the right primitive for a heterogeneous fleet.

Pipeline parallelism inverts the mismatch. Partition the model into contiguous *layer ranges*, one range per node; a token’s hidden state flows through node 0’s layers, crosses the wire once to node 1, flows through its layers, and so on. The consequences line up against the strains almost one-to-one:

- **Activations-only on the wire.** What crosses a node boundary is a single hidden-state tensor, once per boundary, forward — not an `all_reduce` of every layer’s output twice per block. On a ≈ 280 MB/s link that is the difference between a per-token tax of $O(1)$ small transfers and $O(\text{layers})$ full-width collectives; it is the same $\approx 3.3\times$ tax converted from per-layer to per-node.
- **One canonical cast, not a per-op reinterpretation.** The activation is a single tensor at each boundary, so agreeing on one wire dtype (§5.1) is one cast at one seam per node, not a property every collective must carry. The heterogeneous-dtype hazard shrinks from pervasive to pointwise.
- **Format-agnostic.** Activations are just floats; the boundary does not know or care whether the node that produced them ran a safetensors or a GGUF model, on which vendor, with which kernel. Pipeline parallelism sidesteps the format coupling of §5.3 for free, so a mixed cluster can run quantized shards.
- **One generation loop, stateless workers.** The head node runs the single sampling/generation loop; the others are stateless layer-executors that receive an activation, run their layers (holding only their own KV cache), and forward. There is no second generation loop to keep in phase and no cross-node per-token lockstep, so the CUDA-graph asymmetry (§5.3) becomes a purely *local* choice — each node may capture graphs for its own layers or not, and nobody desynchronizes — and the sampling divergence collapses to the head’s single sampler.

Pipeline parallelism is not a free lunch — for a single decode stream it adds pipeline-bubble latency proportional to the number of stages, and throughput needs micro-batching to fill the pipe [9]. Its wide-area latency budget — one network hop per stage on the critical path of every token — is the subject of the companion latency analysis [5], which shows the budget is spendable precisely because pipelining overlaps transport under compute and speculative decoding amortizes the round-trip. But its *coupling* to the cluster is minimal and its assumptions are the ones a heterogeneous fleet can actually honour. The right primitive for one-model-across-many-vendors is the one that puts the least on the wire and demands the least agreement between ranks, and on both axes that is pipeline, not tensor, parallelism. The cross-vendor decode result (§4) proves the *channel* works across the boundary; the argument here is about which *traffic pattern* to run over it, and it is under construction in the same codebase.

7 Reproducibility

Hardware and link. *evo*: AMD Ryzen AI Max+ 395 “Strix Halo,” Radeon 8060S iGPU (gfx1151, RDNA3.5), ROCm. *spark*: NVIDIA GB10 Grace-Blackwell (sm_121), CUDA. *dbc*: Apple M4 Max, Metal. *evo*↔*spark* is a direct 2.5GbE cable (static 192.168.77.0/24), whose measured sustained-TCP goodput is ≈280 MB/s (≈2.24 Gbit/s); its wide-area latency budget is analysed in the companion [5]. The three-node substrate is a wired ethernet mesh with *spark* kernel-IP-forwarding the closing *evo*↔*dbc* edge, so no switch or router is required.

Build. Each node builds the *same* engine source with its own backend feature plus `ring`: `--features "cuda ring"` on *spark*, `--features "rocm ring"` on *evo*. The three mechanical fixes are at commit 63cc4c7ac; the canonical-f32 wire and the `world_size-1` reduce are on `main` (hanzo-quant/src/distributed/, hanzo-engine/src/ops.rs).

Ring configuration. The ring backend is activated by pointing `RING_CONFIG` at a JSON file per node; rank *r* binds `port` and dials its right neighbour at `right_ip:right_port`. A two-node *evo*(rank 0)+*spark*(rank 1) ring:

Listing 2: `RING_CONFIG` for a 2-node cross-vendor ring. Rank 0 (*evo*) listens on 9000 and dials *spark*; rank 1 (*spark*) listens on 9001 and dials back to *evo*, closing the ring.

```
// evo (rank 0, 192.168.77.1): RING_CONFIG=/etc/hanzo/ring0.json
{ "master_ip": null, "master_port": 9002,
  "port": 9000, "right_ip": "192.168.77.2", "right_port": 9001,
  "rank": 0, "world_size": 2 }

// spark (rank 1, 192.168.77.2): RING_CONFIG=/etc/hanzo/ring1.json
{ "master_ip": "192.168.77.1", "master_port": 9002,
  "port": 9001, "right_ip": "192.168.77.1", "right_port": 9000,
  "rank": 1, "world_size": 2 }
```

A `hanzo cluster init / hanzo cluster join <token>` pair wraps this JSON in a scan-to-join token (base58 of the topology plus a terminal QR code) so the operator never hand-writes the ports; the token bakes in the peer list and each worker self-assigns its rank. Both ranks launch the ordinary `hanzo serve` with `RING_CONFIG` set and the same model.

What is measured vs. open. Measured on the real 2.5GbE link: cross-vendor build, connect, `world_size=2` shard, and coherent multi-token *decode* across the AMD/NVIDIA boundary (50.4 tok/s decode, TTFB 0.09s, 213 tok/s prefill on Qwen3-0.6B); the same-vendor `f16+f16` control matching token-for-token; and the single-node *spark* control (168 tok/s decode, 464 tok/s prefill). Open (§5.3, [7]): head-broadcast sampling to close the determinism boundary, $N>2$ cross-vendor runs, GGUF-path TP, and the migration to pipeline parallelism. No projected or synthetic number appears in this paper.

8 Related work and contribution

Tensor parallelism is Megatron-LM [8]; pipeline parallelism is GPipe and its successors [9]; the on-device collectives are NCCL [11] and RCCL [12], each single-vendor by design. Distributed inference over commodity/heterogeneous networks has been explored most prominently by Petals [10], which runs pipeline-parallel shards across volunteer nodes over the public internet — close in spirit to §6,

and evidence that the activations-only pattern is the one that survives a slow, loose network. What we add is narrow and, to our knowledge, not previously shown: a *single* inference engine, one source lowered to three vendors’ kernels, running a *tensor-parallel* forward *and coherent decode* across an *AMD+NVIDIA* boundary on *commodity ethernet* with *no* shared-vendor collective — and an unusually explicit account of the failure modes that a host-staged cross-vendor collective exposes (the 1en^2 overflow, the unbuilt ROCm arm, the startup-race timeout, and the heterogeneous-dtype wire). The design contribution is the observation, made concrete by those failures, that heterogeneity is a stress test for homogeneity assumptions, and that pipeline parallelism minimizes exactly the assumptions that break. This paper is the distributed extension of the “one source, every backend” thesis [1, 2]: having made one engine run on every backend, we make those backends run in one cluster. Two companions extend the arc: the latency-physics analysis [5] carries the wide-area budget for §6, and the three-plane compute-fabric blueprint [6] turns the trusted-LAN ring into a permissionless, consensus-managed, PQ-authenticated network. The open questions this result raises are collected as a research agenda in the open-problems companion [7].

9 Conclusion

No collective library crosses a vendor boundary, so a mixed AMD/NVIDIA/Apple fleet cannot run one model together through the standard path. Host-staging the collective through CPU memory and TCP makes the channel vendor-agnostic, and on that channel one engine — built from a single source for three vendors’ kernels — shards a model `world.size=2` across an AMD/NVIDIA boundary over a 2.5GbE cable and *decodes coherently* at 50.4 tok/s, which we believe is a first. The road to that result ran through three bugs in a path that compiled for months but had never run — a reminder that “it builds” certifies almost nothing about a distributed system — and one deeper hazard, a heterogeneous-dtype wire that a same-vendor cluster can never expose, cured by a canonical `f32` payload and an accumulating `world.size-1` all-reduce correct for any $N \geq 2$. The measured $\approx 3.3\times$ synchronization tax, and the residuals that remain — local sampling that can diverge at a near-tie, graphless lockstep between two generation loops, and TP’s coupling to the dequantized weight path — all point one way: toward pipeline parallelism, whose activations-only, one-generation-loop, format-agnostic shape asks of a heterogeneous cluster only what such a cluster can actually give. The channel is open across the boundary and coherent over it; the companions drive the right traffic [5], harden it into a network [6], and enumerate what is left to solve [7].

References

- [1] Hanzo AI. *Edge LLM Inference Across Commodity Accelerators: Measured AMD, NVIDIA, and Apple Performance, and hanzo-engine at llama.cpp Decode Parity on Every Backend*. Paper 05.
- [2] Hanzo AI. *One Source, Every Backend: The hanzo-kernel DSL — Thesis and Guided Tour*. Paper 07 (umbrella).
- [3] Hanzo AI. *Fusion Is Composition: Kernel Fusion as the Functor Law*. Paper 07.1.
- [4] Hanzo AI. *One Oracle, Every Backend: The CPU Bit-Exactness Law*. Paper 07.4.
- [5] Hanzo AI — ML Systems. *The Latency Physics of Geo-Distributed Autoregressive Decode*. Distributed-inference companion (analytical).
- [6] Hanzo AI — ML Systems. *The Three-Plane Compute Fabric: Consensus-Managed, ZAP-Transported Decentralized AI Inference*. Distributed-inference companion (architecture).

- [7] Hanzo AI — ML Systems. *Open Problems in Heterogeneous Distributed Inference*. Distributed-inference companion (research agenda).
- [8] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, B. Catanzaro. *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism*. arXiv:1909.08053, 2019.
- [9] Y. Huang et al. *GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism*. NeurIPS, 2019.
- [10] A. Borzunov et al. *Petals: Collaborative Inference and Fine-tuning of Large Models*. arXiv:2209.01188, 2022.
- [11] NVIDIA. *NCCL: NVIDIA Collective Communications Library*.
- [12] AMD. *RCCL: ROCm Communication Collectives Library*.